

Project 4: Data Classification

Introduction

The objective of this project is to perform a complete descriptive analysis and required preprocessing on the “moocs.xlsx” dataset, to then classify and predict with respect to Urgency, Confusion, and Sentiment. The process includes a long process of Initial analysis, data cleaning, feature engineering, visualization, text preprocessing and comparing different models. We compare the three suggested text representations: TF, TF-IDF, and Sentence Embeddings (applying the appropriate text cleaning for each), then comparing them using a light baseline model, the one we choose was Logistic Regression.

After Finding the best text representation, we use it to compare the five given models with respect to the three classes, choosing the best two models for each. Then finally applying grid search to optimize selected hyperparameters for each of those models, comparing their score, to select the winning model (for each target feature).

Initial Data Exploration

We first inspect the general structure of the dataset using `.head()`, `.info()`, and `.describe()`, from those we can see that we have 18 features, and 29604 rows. We can also see that the first and main feature, “Text”, is a text raw form of what appears to be comments on the “moocs” platform, which will need a lot of preprocessing (as we will get into later), we can also see that the 3 target features need to be binned into binary classes, since their values range from 1-to-7, when our goal is to predict whether it is true or false.

We can also see features like: “CourseType”, “course_display_name”, and “post_type” need to be encoded as the models we are using are mathematical and only take numbers.

Also features like “reads” and “up_count” need to be binned aswell, while other features need to be dropped. We also need to extract features from the “created_at” feature to help our model predict more accurately, handle missing values, and handle duplicates.

All of which have be applied in the correct manner and order to ensure no errors in the data, as we will get into in the following sections.

Data Cleaning

In this section go through some steps, which are:

- **Dropping Useless Features:** features like 'forum_uid', 'forum_post_id', and 'comment_thread_id', have no real use as they give no meaning and would only hinder the models, same with the 'anonymous_to_peers' feature, it is all zeros so it is completely useless. We handle them by dropping all four.
- **Handle Missing Values:** from the earlier inspection, we saw that there were a couple of columns with missing values, which would crash our models, and since they are not a lot (only a small fraction of dataset, approx. 21 rows), it is better to drop them, as using something like KNN imputer to fill the missing data for a text dataset like this would give terrible results, so we drop all the rows with missing data.
- **Handle Duplicates:** We check for duplicates in the text column alone, as same comments can be made at different times, to do so we must convert it to string format first, then check for duplicates. We found 478 duplicate texts, which we will handle by dropping their whole rows, leaving only one occurrence of each duplicate, to prevent overfitting.

Feature Engineering

Using graphs and plots for better understanding, we observe that:

- **Reads and Up Count** have very high skews, but instead of capping or applying log transformations, we revert to binning instead, which is better as it will preserve the value of the feature, improving the models' performance. We use qcut to split reads into 3 equal bins, low, medium and high, then we map them from 0 to 2, since the level of reads gives actual weight, and for the up count, we only need to make 1 new binary column, is_upvoted, indicating whether a post was upvoted or not, as most of the values are zero.
- **CourseType, post type and course display name** all need to be one hot encoded into numerical values, which are accepted by our model. We choose one-hot encoding as different course names do not have higher weights.

- **Urgency, Confusion and sentiment** need to be binned into binary classes as we stated earlier, so that we can predict whether a post is urgent/confused/positive or not. We choose to split them all down the middle (≥ 4 is true else is false).
- Meanwhile, **Created at** feature needs a completely different approach. We first start by observing distribution of post across the day by the hour, by observing the line plot we can see deduce a time of day feature, to see how daytime affects posting behavior, second we plot the timeline of posts per course, which shows us that courses tend to have a spike when they first open, and then on a downfall trend, third, we plot the distribution of posts across days of the week, from this plot we can deduce another feature to see if weekdays have higher different effect than weekends.

We Finish off by dropping all the old features that have already been modified.

Visualization and EDA

We check the distributions of the target features to check for data imbalance, By analyzing the box plots, we can see that urgency and sentiment classes have very uneven distributions, which we handle later when setting the hyperparameters of the models. We also inspect the distribution of all the new features using histograms to double check.

Text Preprocessing

The following step is to handle the text feature, we will be testing three different text representation, which are: TF, TF-IDF, and Sentence Embeddings, the first two needing some heavy cleaning, while the last barely needs any cleaning, as it requires context. But first we have to make 3 copies for each type, then apply train-test splits on them (80-20 as instructed). To prepare the text for TF and TF-IDF, we have to clean them first, after inspection of the comments, we deduced the following needed steps for cleaning: convert to lowercase, remove html tags, hex values, URLs, punctuation, and

standalone numbers, we then apply `WordNetLemmatizer()` to clean the text. By testing the function on a sample text we see that it works perfectly.

Now we move on to the TF and TF-IDF pipelines, since they are very similar. They have to go through the following phases : text cleaning using lemmatizer, applying their respective models (`CountVecorizer` and `TfidfVectorizer` for TF and TF-IDF respectively), and the most important step is to apply `TruncuatedSVD` to reduce the number of feature from the large original number which would need a lot of computational power, to a much more reasonable number of features.

For Embeddings, we only apply light cleaning by removing tags, hex values, and urls, leaving the rest as is for the model to work with. The embedding pipeline goes as follows: first text cleaning, then we apply the embeddings model, for out case we used the lightweight “all-MiniLM-L6-v2” model, as it is perfect for our application.

Baseline Model Evaluation

Now that we have all 3 representations ready, we picked the lightest model with the simplest setup to compare the result of all 3 across the 3 different target features, resulting in 9 different models. The model of choice was Logistic Regression (we insure to set `class_weight = 'balanced'` to manage the imbalance), by running all 9 of the models, calculating and comparing their accuracy and F1 scores, we can see that the best representation for all 3 target features is Embeddings, so we are going to use it for our final model comparison. Results:

BASELINE ACCURACY (%)

Text Representation	1. TF	2. TF-IDF	3. Embeddings
Target Variable			
Confusion_Class	82.13	82.56	82.80
Sentiment_Class	74.51	76.09	79.11
Urgency_Class	88.89	89.18	89.35

BASELINE F1-SCORE (%)

Text Representation	1. TF	2. TF-IDF	3. Embeddings
Target Variable			
Confusion_Class	82.49	82.93	83.18
Sentiment_Class	77.77	79.09	81.56
Urgency_Class	89.19	89.48	89.69

Final Model Comparison

Now that we the data is ready, and we have chosen embedding, we will compare 5 different models for each target feature, picking the best 2 to apply grid search to and test to find the best mode. The five models we will be comparing are: Naïve Bayes, SVM, Random Forest, AdaBoost, and Logistic Regression. After making sure we set class_weight to balanced, we run all 15 models on their basic settings, calculate their score, and compare them. By doing so we find that the best models for each target feature are:

- **Urgency:** AdaBoost and SVM
- **Confusion:** SVM and Logistic Regression
- **Sentiment:** SVM and AdaBoost

Results:

FINAL ACCURACY LEADERBOARD (EMBEDDINGS)

Target Model	Confusion_Class	Sentiment_Class	Urgency_Class
1. Naïve Bayes	80.23	73.63	76.69
2. SVM	83.75	82.37	89.69
3. Random Forest	80.98	86.33	89.11
4. AdaBoost	81.05	86.43	90.64
5. Logistic Regression	82.80	79.11	89.35

FINAL F1-SCORE LEADERBOARD (EMBEDDINGS)

Target Model	Confusion_Class	Sentiment_Class	Urgency_Class
1. Naïve Bayes	80.05	77.04	78.50
2. SVM	84.11	84.17	90.00
3. Random Forest	80.10	81.94	88.03
4. AdaBoost	80.91	83.87	90.39
5. Logistic Regression	83.18	81.56	89.69

Now, we have to optimize the hyperparameters, but only the ones that will give actual useful difference to the accuracy without too much computational power.

Hyperparameter Optimization using GridSearch

For the finally, we have 3 remaining models, each of which we will only pick certain hyperparameters to optimize, as it does not make sense to waste hours for a slight improvement. The models are:

1. **AdaBoost:** the hyperparameters we are going to focus on are `n_estimators` and `learning_rate` as they are the most important and useful for this case. For `n_estimators` we will be testing the following values [50, 100, 200], and for `learning_rate` the testing values are [0.1, 0.5, 1.0], as we are trying to achieve the best performance without too much learning time.
2. **SVM:** as for this one, we will be using gridsearch for `C` and `kernel`, the first of which is a penalty parameter, and the second controls the shape of the line splitting the classes. For `C` the values we will use are [0.1, 1, 10], and for `kernel` ['linear', 'rbf'].
3. **Logistic Regression:** we will only be focusing on `C` and `solver` hyperparameters, as we explained above. For the `C` the values we will use are [0.1, 1, 10], and for `solver` ['liblinear', 'lbfgs'].

Conclusion

After running gridsearch, for each model, with respect to the classes they are applied on, and comparing all their scores, we see that the clear winner for all three classification models is the SVM, with the best parameters for urgency and sentiment being `c: 10` and `kernel rbf`, while confusion had the same kernel but `c: 1`. By Inspecting the following table we can note and compare the score for all 3 models, the highest being the urgency class, and the lowest being the confusion class. Scores for Best SVM Models:

	Urgency	Confusion	Sentiment
Accuracy:	91.26%	83.75%	86.75%
Precision:	91.26%	85.51%	86.75%
Recall:	91.26%	83.75%	86.75%
F1-Score:	91.36%	84.11%	86.75%
'C':	10	1	10